PYTHON NOTES

VARIABLES

A variable is like a container that holds a value. Python is dynamically typed, meaning you `don't have to declare the variable's type` when you define it.

DATA TYPES

- `Integer (int)` // By default integeres are big
- `String` // Can be used as 'Str', "Str", '''Str'''
- `float` // number with decimal point
- `boolen (bool)` represent True/False
- `None` // There is no double

```
name = "Aanshik"
age = 20
height = 5.6
print(type(name), type(age), type(height) )

#OUTPUT
# <class 'str'> <class 'int'> <class 'float'>
```

Type Conversion Vs Type Casting

- Type Conversion - implicit - small to big - done by compiler
- **Type Casting** - explicit - big to small - done by user

```
price = 75.56
percent = price + 5 # Conversion
print(percent)
# Output: 80.56

price = 75.56
percent = int(price) # Casting
print(percent)
# Output: 75
```

COMMENTS

- `#` Singe Line Commments
- `""" """` Multi line Comments

PRINT STATEMENT

```
print(object(s), sep=' ', end='\n', file=sys.stdout, flush=False)
```

```
print("Hello World", 45, true)
print(10, 20, sep='-', end=' | ')
print(30, 40, sep='+',file=sys.stdout)
# Output: 10-20 | 30+40

import time
print("Starting process...", end='', flush=True) # Forces immediate display
time.sleep(2)
print(" Done.")
```

- `objects(s)` # single or multiple objects which are printed
- `sep=' '` # it is the connector between objects, by default ' '
- `end='\n'` # it tell what to print after last object, by default '\n'
- `file=sys.stdout` # it tells where the output should be written, by default sys.stdout i.e. console
- `flush=false` # It forces output buffer to be written immediately, by default false, allowing OS to manage

F-Strings

While print() handles basic output, a cleaner and more powerful way to format and print complex strings is using f-strings (Formatted String Literals)

F-strings allow you to embed expressions and variables directly inside string literals by prefixing the string with f or F

```
item = "coffee"
price = 3.50

# Using print() with commas:
print("The", item, "costs", price)

# Using an f-string (cleaner, faster):
print(f"The {item} costs ${price:.2f}.")
# Output: The coffee costs $3.50.
```

OPERATORS

Arithematic Operators

`+`, `-`, `*`, `/`, `%`, `**` (power), `//` (floor division)

Relational Operators

`==`, `!=`, `<`, `>`, `<=`, `>=`

Logical Operators

`and`, `or`, `not`

Unary Operators

Python does not support any unary operators

Bitwise Operators

`&`, `|`, `<<`, `>>`, `^`, `~`

Assignment Operators

`=`, `+=`, `-=`, `*=`, `/=`, `%=`, `**=`

USER INPUT

```
num = input("Enter a number: ")
# input data is always a string

num = int(input("Enter a number: "))
# type casted to integer
```

CONDITIONAL STATEMENTS

Conditional Statements are used to decide to do something based on some condition.

```
if condition:
    # code
elif condition:
    # code
else:
    # code
```

> NOTE : Python can have indentation error

LOOPS

- Loops are used to iterate through a block of code multiple times.
- `Break` statement is used to break out of the loop.
- `Continue` statement is used to skip the current iteration of the loop.
- `pass` statement is used to do nothing.

```
while condition:
    # code
```

```
for item in list:
    # code
```

```
for item in range(start, end, step):
    # code
```

```
str = "Aanshik"
for char in str:
    if (char == 'i'):
      print("Found")
      break
else:
  print("Not Found") # code runs when loop runs completely
```

- `range(start = 0, end, step = 1)` // creates a sequence of numbers from start to end-1 with step

```
print(range(5)) # range(0,5)
seq = range(1,10,2)
print(seq) # range(1,10,2)
print(seq[1]) # 3
print(list(seq)) # [1,3,5,7,9]
range(5, 0 , -1) # [5,4,3,2,1]
```

FUNCTIONS

Functions are blocks of code that perform a specific task which can be reused multiple times in a program, hence reducing redundancy.

- we can return multiple values from a function

```
def prodSum(a, b):
    return a + b, a * b

print(prodSum(3, 2)) # (5, 6)
```

- we can set the default values for the parameters

```
# def sum(a = 2, b): # Error
def Sum(a, b = 2):
    return a + b
print(Sum(3)) # (5, 6)
print(Sum(5,3)) # (8,15)
```

STRINGS

Strings can be created by wrapping with `''` or `""` or `''' '''` or `""" """`

```
# name = 'Aanshik's Phone' # Wrong
name = "Aanshik's Phone" # Right
# different ways are to distinguish between single and double quotes
```

String Methods

- `Concatenation` // can be done useing `+`
- `len()` // gives length of the string
- `Repetition` // can be done using `*`

- `str[i]` // gives character at index i

  > NOTE : we cannot change the ith char using this, strings are immutable

- `str[i:j]` // gives substring from i to j-1
- `str[i:j:k]` // gives substring from i to j-1 with step k
- `str[i:]` // gives substring from i to end
- `str[:j]` // gives substring from start to j-1
- `str[-i:-j]` // gives substring from -i to -j-1, minus indexing is allowed and starts from end with -1
- `str[::-1]` // gives reversed string

- `endswith("word")` // checks if string ends with word
- `startswith("word")` // checks if string starts with word
- `replace("old", "new")` // replaces all old with new
- `find("word")` // gives index of first occurence of word else -1
- `count("word")` // gives count of word
- `index("word")` // gives index of first occurence of word else error
- `rfind("word")` // gives index of last occurence of word else -1

- `capitalize()` // capitalizes first letter
- `islower()` // checks if string is lowercase
- `isupper()` // checks if string is uppercase
- `lower()` // converts to lowercase
- `upper()` // converts to uppercase
- `center(width)` // centers string with width
- `isnumeric()` // checks if string is numeric

- `title()` // capitalizes first letter of each word
- `swapcase()` // converts lowercase to uppercase and vice versa
- `strip()` // removes leading and trailing spaces
- `split("separator" = " ")` // splits string into list of words`
- `join()` // joins list of words into string
- `join(separator)` // joins list of words into string with separator

LISTS and TOUPLE

- Lists are similar to arrays, but they are mutable while Tuples are immutable, once created cannot be changed.
- There can be different data types in a list
- Lists are enclosed in square brackets `[]` and Tuples are enclosed in round brackets `()`

```
list = [1,"Python", 3]
tuple = (1, 2, "CPP")
print(list) # [1, "Python", 3]
print(tuple) # (1, 2, 3)
```

- we can access the elements of the list and tuple using index

```
print(list[0]) # 1
print(list[1]) # "Python"
print(tuple[2]) # CPP
```

```
tup = (1,) # Tuple with single element
tup = (1) # Integer
tup = () # Empty tuple is also valid
```

List Methods

- `append()` // adds element to the end of the list
- `pop()` // removes last element
- `remove(element)` // removes first occurence of element

- `sort()` // sorts the list
- `sort(reverse=True)` // sorts the list in reverse order, Sorting is not only done with numbers but with strings as well

```
list = ["S", "V", "A", "b", "z"]
list.sort() # ['A', 'S', 'V', 'b', 'z']
list.sort(reverse=True) # ['z', 'V', 'S', 'b', 'A']
```

- `insert(index, element)` // inserts element at index
- `clear()` // removes all elements
- `index(element)` // returns index of first occurence of element
- `count(element)` // returns count of element
- `reverse()` // reverses the list

- `list.copy()` // returns a shallow copy of the list, not avaliable for the Tuple

```
ls = [1, 2, 3]
list2 = ls.copy() # shallow copy created, a new list is created but the element objects are shared
print(list2) # [1, 2, 3] both list point to the same object
ls[0] = 4 # it does not affect the other list, only first pointer points to new integer object 4
print(ls) # [4, 2, 3]
```

Slicing in list is similar to the strings

```
list = [1,2,3,4,5]
print(list[1:]) # [2, 3, 4, 5]
print(list[:3]) # [1, 2, 3]
print(list[1:3]) # [2, 3]
print(list[::2]) # [1, 3, 5]
print(list[::-1]) # [5, 4, 3, 2, 1]
```

Tuple Methods

- `count(element)` // returns count of element
- `index(element)` // returns index of first occurence of element

DICTIONARY

- A dictionary is a collection of key-value pairs.
- They are enclosed in curly braces `{}`
- They are unordered, mutabe, and do not allow duplicate keys.
- They are indexed by keys, which can be of any immutable type.
- keys cannot be list or dictionary, but values can be any data type

```
dict = {
  "name": "Aanshik",
  "marks": {
    "Physics": 98,
    "Chemistry": 95,
    "Maths": 96
    },
  18 : true
  }
print(dict) # {'name': 'Aanshik', 'marks': {'Physics': 98, 'Chemistry': 95, 'Maths': 96}, 18: True}
dict["name"] = "Vinay"
print(dict["name"]) # Vinay
```

Dictionary Methods

- `keys()` // returns a list of keys
- `values()` // returns a list of values
- `items()` // returns a list of tuples of key-value pairs

```
dict = {
  "name": "Aanshik",
  18 : true
  }
print(dict.keys()) # dict_keys(['name', 18])
print(dict.values()) # dict_values(['Aanshik', True])
print(dict.items()) # dict_items([('name', 'Aanshik'), (18, True)])
print(list(dict.keys())) # ['name', 18] dict keys type casted to list
```

- `dict["key"]` returns value, error when does not exist
- `get(key, default=None)` // returns the value of the key if it exists, otherwise none
- `pop(key, default=None)` // removes and returns the value of the key if it exists
- `update(dict)` // updates the dictionary with the key-value pairs from another dictionary
- `popitem()` // removes and returns last key-value pair

SET

- Set is a mutable collection of unordered, unique and immutable objects
- we cannot have list, and dictionary in set as they are mutable
- Set is enclosed in curly braces `{}`

```
set = {1, 2, 3, 4, 5}
print(set) # {1, 3, 2 , 5 , 4} ordered is not maintained
```

Set Methods

- `add(element)` // adds element to set
- `remove(element)` // removes element from set, error if not present
- `discard(element)` // removes element from set if present
- `pop()` // removes and returns an arbitrary element from the set
- `clear()` // removes all elements from the set
- `copy()` // returns a shallow copy of the set
- `set1.union(set2)` // returns a union of sets
- `set1.intersection(set2)` // returns an intersection of sets

```
set = {1, 2, 3, 4, 5}
print(set) # {1, 3, 2 , 5 , 4}
set.add(6)
print(set) # {1, 3, 2 , 5 , 4, 6}
set.remove(6)
print(set) # {1, 3, 2 , 5 , 4}
set.discard(6)
print(set) # {1, 3, 2 , 5 , 4}
set.pop()
print(set) # {1, 3, 4 , 5 }
set.clear()
print(set) # set()
```

FILE HANDLING

File handling is a process of creating, reading, updating and deleting files in a computer system.

- `open(file, mode = 'r')` // creates a file object

```
f = open("file.txt", "r")
# MODES
# 'r' - read mode (default)
# 'W' - write mode(overwrites)
# 'a' - append mode
# 'x' - create mode
# 't' - text mode (default)
# 'b' - binary mode
# '+' - update mode (read/write)
# 'w+' - write and read mode
# 'a+' - append and read mode
```

> if a non existing file is opened in `a` or `w` mode then it is created

---

- `close()` // closes the file object
- `read()` // reads the entire content of the file
- `readline()` // reads a single line from the file
- `write(string)` // writes a string to the file
- `seek(offset, from)` // moves the cursor to a specific position in the file

```
f = open("file.txt", "r")
print(f.read()) # prints entire content
f.close()

f = open("file.txt", "r")
print(f.readline()) # prints 1st line
f.close()

with open("file.txt", "r") as f:
    print(f.read())
# with closes the file automatically
```

OS MODULE

- `os.remove(file)` // deletes a file
- `os.rename(old_file, new_file)` // renames a file
- `os.mkdir(directory)` // creates a directory
- `os.rmdir(directory)` // removes a directory
- `os.chdir(directory)` // changes the current working directory
- `os.getcwd()` // returns the current working directory

OOPS

OOPs is a programming model that provides a way to structure programs by grouping related data and behavior into objects.

- `class` // It is the blueprint of an object
- `object` // It is a real instance of the blueprint
- `self` // refers to the current object

```
class Person:
    def __init__(self, fullname, age):
        self.name = fullname
        self.age = age

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)

person = Person("Aanshik", 20)  # object
person.display()
```

Constructor

- all classes have a default constructor called `__init__` We can write custom cunstructors by redefining it
- Constructor method takes a compulsory parameter `self` as its first argument
- `self.var` creates the instance variables

> NOTE : Self is just a variable name of first parameter, we can change it to some other name

```
# default constructor
def __init__(self):
  pass

# parameterized constructor
def __init__(self, name):
  self.name = name

# Same class can have multiple constructors, this is called method overloading
```

Variables

- Instance variables are those that are unique to each instance of a class
- Instance variables are created at object creation using `self` parameter
- Class variables are those that are shared by all instances of a class

```
class Student:
  College = "IIITG" # class variable
  def __init__(self, name, roll):
    # instance variable
    self.name = name
    self.roll = roll
```

Methods

Methods are functions defined inside a class

- They are of three types
  - Instance methods
  - Static methods
  - Class methods

INSTANCE METHODS

```
class Student:
  def __init__(self, name, roll):
    self.name = name
    self.roll = roll

  # self is the compulsory parameter
  def display(self):
    print("Name:", self.name)
    print("Roll:", self.roll)
```

STATIC METHODS

Static method is:

- Just a normal function inside a class
- Has no access to class or instance (self, cls)
- Used for utility logic related to the class
- Defined using the `@staticmethod` decorator

```
class Math:
  @staticmethod
  def add(a, b):
    return a + b
```

CLASS METHODS

Class method is a method that:

- Works with the class itself
- Has access to class variables directly
- Uses `cls` as the first parameter
- Defined using the `@classmethod` decorator

```
class Student:
  college = "IIITG"

  @classmethod
  def change_college(cls, new):
    cls.college = new
```

> NOTE
>
> ```
> class student:
>  college = "NONE"
>
>  def __init__(self, name, college):
>   self.name = name
>    # Accessing class variable indirectly
>    student.college = college
>    self.__class__.college = college
>
>  # Accessing class variable directly
>  @classmethod
>  def change_college(cls, new):
>    cls.college = new
> ```

> NOTE : `decorators` allow us to wrap another function in order to extend the behaviour of the wrapped function Without permanently modifying it, Ex - @propery, @getter, @setter, @classmethod

---

@property Decorator

```
class Result:
  def __init__(self, Ph, Ch, Ma):
    self.Ph = Ph
    self.Ch = Ch
    self.Ma = Ma
    # percent = (Ph + Ch + Ma) / 300 * 100
    # Not updated when marks change
  @property
  def percent(self):
    return (self.Ph + self.Ch + self.Ma) / 300 * 100

r = Result(98, 97, 99)
print(r.percent) # 98
r.Ma = 96
print(r.percent) # 97
```

Abstraction

Abstraction is the process of hiding the implementation details and showing only the essential features of an object to the user.

Encapsulation

Encapsulation is the process of wrapping data and the methods that work on data within a single unit.

Del Keyword

- `del` keyword is used to delete an object or its properties

```
class Student:
  def __init__(self, name, roll):
    self.name = name
    self.roll = roll

  def display(self):
    print("Name:", self.name)
    print("Roll:", self.roll)

s = Student("Aanshik", 20)
del s.display() # deletes display method
del s # deletes object
```

Access Modifiers

Modifier	Syntax	Meaning
Public	`name`	Accessible everywhere
Protected	`_name`	Accessible within the class or child
Private	`__name`	Accessible within the class

Inheritance

Inheritance is the process of creating a new class from an existing class.

```
class Engine:
  def __init__(self, cc):
    self.cc = cc

  def display(self):
    print("CC:", self.cc)

class Car(Engine):
  def __init__(self, cc, name):
    super().__init__(cc)
    self.name = name

ch = Car(1000, "Lamborghini")
ch.display() # CC: 1000
```

- `super()` is used to access the parent class
- `super().__init__(name)` is used to call the parent class constructor

Polymorphism

Polymorphism is the ability of objects to take on many forms

- `Method Overloading` : same method name with different parameters

```
class Student:
  def display(self, name, roll):
    print("Name:", name)
    print("Roll:", roll)

  def display(self, name):
    print("Name:", name)

s = Student()
s.display("Aanshik", 20) # Name: Aanshik, Roll: 20
s.display("Aanshik") # Name: Aanshik
```

- `Method Overriding` : same method name with same parameters

```
class Parent:
  def display(self):
    print("Parent's display method")

class Child(Parent):
  def display(self):
    print("Child's display method")

p = Parent()
c = Child()
p.display() # Parent's display method
c.display() # Child's display method
```

OPERATORS OVERLOADING

- Dunder methods are methods starting with `__` and ending with `__`
- `__add__` is used to overload the `+` operator
- similarly,
  - `__sub__` for `-`
  - `__mul__` for `*`
  - `__truediv__` for `/`
  - `__mod__` for `%`
  - `__pow__` for `**`

```
st = "Py" + "thon"
n = 2 + 3
ls = [1, 2] + [3, 4]
# same `+` operator work differently
```

```
class pairSum:
  def __init__(self, a, b):
    self.a = a
    self.b = b

  def __add__(self, other):
    return self.a + other.a, self.b + other.b

p1 = pairSum(1, 2)
p2 = pairSum(3, 4)
print(p1 + p2) # (4, 6)
```

Destructor

- `Destructor` : all classes have a default destructor called `__del__`